

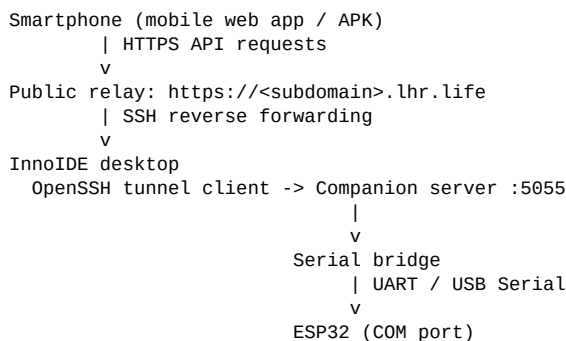
InnoIDE Cloud Remote Tunneling Architecture

IoT Companion Studio • localhost.run (lhr.life) • OpenSSH reverse forwarding

Purpose. This document describes how InnoIDE connects a mobile companion to an ESP32 connected by USB Serial to the desktop, using an automatically managed public SSH tunnel.

1. Overview

InnoIDE's IoT Companion Studio enables users to control, monitor, and automate ESP32 microcontrollers from a smartphone. The desktop hosts a local companion server and a serial bridge; a reverse SSH tunnel exposes the local server through a public HTTPS hostname.



The relay hostname is a network endpoint, not the project's permanent identity. With anonymous ephemeral tunnel allocation, the hostname may change after reconnection. A QR code containing that hostname can therefore become stale.

2. Why use localhost.run (lhr.life)?

2.1 Remote access over cellular and WAN

A phone on 4G/5G or a different network cannot normally reach a desktop's private LAN address. Local Wi-Fi access may also be blocked by client isolation or network firewall rules. A reverse tunnel creates an outbound connection from the desktop to a public relay, allowing the phone to reach the desktop through the relay without requiring inbound router port forwarding.

2.2 Low-friction setup with OpenSSH

The design uses the OpenSSH client available on many Windows 10/11 installations, avoiding a separate tunnel client in the ordinary case. Anonymous forwarding can reduce setup steps, but availability, provider limits, host-key handling, and service terms remain external dependencies.

2.3 HTTPS and browser security

The public endpoint is HTTPS, which supports secure browser features such as camera access for QR scanning and avoids mixed-content requests when the companion is served over HTTPS. TLS protects the phone-to-relay leg. The SSH tunnel protects the relay-to-desktop leg. This does not remove the need for authentication and authorization in the companion APIs.

3. Execution flow and lifecycle

1. Start local companion server
Node.js HTTP server listens on 127.0.0.1:5055
2. Start SSH reverse tunnel

```
ssh -R 80:127.0.0.1:5055 nokey@localhost.run
```

3. Receive allocated public hostname
`https://<subdomain>.lhr.life`
4. Synchronize tunnel state
 - Write `active_tunnel.json`
 - Notify renderer: `companion:tunnel-status`
 - Display the current URL / QR code
5. Mobile scans QR code and opens companion
6. Mobile action
`POST /api/action { "payload": "LED:1" }`
7. Local server dispatches serial command
`Serial bridge -> ESP32 -> GPIO action`

The tunnel daemon should treat the provider's returned hostname as live session state. The renderer should display a clear connected, reconnecting, or offline state and avoid presenting a stale URL as active.

4. Network modes

Feature	Cloud Remote	Local Wi-Fi
Endpoint	<code>https://<hash>.lhr.life</code>	<code>http://<LAN-IP>:5055</code>
Reach	Internet / cellular / WAN	Same reachable local network
Setup	Automatic SSH tunnel	Same LAN; firewall permits access
Transport	HTTPS + SSH tunnel	HTTP in the described design
NAT traversal	Outbound tunnel avoids inbound port forwarding	May be blocked by isolation/firewall
Latency	Depends on relay and internet path	Usually lower on a healthy LAN

The latency ranges sometimes used in planning are estimates, not guarantees; actual performance depends on the route, relay load, network conditions, and device workload.

5. Codebase implementation map

File	Responsibility
<code>electron/ipc/index.js</code>	IPC handlers for tunnel start/stop, provider fallback, and status synchronization.
<code>scratch/cloud_tunnel_daemon.cjs</code>	Background supervisor for SSH tunnel lifecycle and reconnect attempts.
<code>scratch/active_tunnel.json</code>	Runtime state: active URL, timestamp, and active status; should be treated as transient.
<code>electron/dev-runner.js</code>	Development supervisor for Vite, Electron, and tunnel daemon processes.
<code>src/components/ESP32Flasher.jsx</code>	Companion publishing UI, QR rendering, network-mode selection, and device synchronization.
<code>main/main.c</code>	ESP32 firmware parsing UART commands such as <code>LED:1</code> / <code>LED:0</code> and controlling GPIO.

6. Redundancy and fault tolerance

6.1 Provider fallback

If `localhost.run` is unavailable or rate-limited, the system may attempt a secondary SSH tunnel provider such as `Serveo`. Each provider can have different hostnames, limits, and behavior; fallback should be reported as a provider change while preserving the project identity.

6.2 Automatic reconnection

When the SSH process exits or connectivity is lost, the supervisor can retry after a short delay. Use bounded retries with backoff, detect laptop sleep/resume, and verify that the new tunnel is actually reachable before marking it connected.

6.3 Stale-state invalidation

The `active_tunnel.json` file is a runtime cache, not durable project data. On startup, validate its timestamp and active state; clear or mark stale entries when the tunnel is no longer reachable. A 15-minute age threshold is a policy choice, not proof that a tunnel is dead.

7. Stable QR codes: separate identity from connection

A QR code is a fixed encoding of its contents. If it contains an ephemeral tunnel URL, it cannot follow a later tunnel URL automatically. To keep QR codes stable, encode a permanent project URL and resolve it to the current connection.

```
Permanent QR URL:
https://companion.example.com/p/<project-id>

Project registry:
project-id -> current tunnel URL + connection status

On tunnel reconnect:
1. Obtain new tunnel URL
2. Update the project registry
3. Keep the permanent QR URL unchanged
```

A stable hostname alone is not sufficient for multiple projects: include a permanent project ID/path and route requests to the right project. For one desktop hosting many projects, one stable public hostname and distinct paths may be enough. For projects on different desktops, a cloud registry and routing layer can map each project to its active desktop.

Redirect vs reverse proxy: A redirect sends the phone to the temporary tunnel URL, so the temporary address remains visible and subsequent requests use it. A reverse proxy keeps the phone on the permanent domain and forwards HTTP/WebSocket traffic to the registered connection, but requires a reachable routing service and careful authentication, timeouts, and WebSocket support.

8. Security and operational considerations

- Treat the QR code as a project locator, not as a password or authorization token.
- Authenticate mobile sessions and authorize every project/device action.
- Validate tunnel URLs and project registration; prevent one project from targeting another project's connection.
- Use HTTPS for public access and avoid exposing unrestricted local APIs.
- Do not disable SSH host-key checking without understanding the security trade-off; use provider-documented host-key verification where possible.
- Provide offline/reconnecting status and a way to revoke mobile access.
- Do not claim the companion is always available: the local desktop, serial bridge, device, network, and relay must be operational.

9. Summary

The current `lhr.life` design provides convenient remote access by forwarding a public HTTPS endpoint through an SSH reverse tunnel to the local Express companion server. Its dynamically allocated hostname is session-dependent, so QR codes containing it are not durable. The durable design is a permanent project URL plus a registry/routing layer that tracks the current tunnel. Electron, Node.js, and Express can run the local companion services, but a stable public URL requires a permanent domain and an always-reachable routing endpoint.